

L2 - 13.4. Dissecting PE Headers

Task 1 - Introduction

An **executable (.exe) file**, also known as a **Portable Executable (PE) file**, contains code designed to run programs on any Windows Operating System. It is based on the Common Object File Format (COFF) data structure, which also encompasses DLLs, Linux shared objects, and ELF files.

Task 2 - Overview of PE headers

On disk, a PE file is stored as a series of bits, which appear as a collection of hexadecimal characters when opened in a Hex editor. These hex characters represent the specific instructions the Windows Operating System reads and runs to execute the binary.

[illegible]

`wxHexEditor` open-source hex editor designed specifically to handle and edit extremely large binary files and disk images.

pe-tree A Python-based GUI tool used in malware analysis to parse, visualize, and inspect the structure of Portable Executable (PE) files in a hierarchical tree view.



A **struct** is a user-defined data type that groups different types of data elements together into a single variable. Because its structure is custom, you need to consult documentation—such as Microsoft's MSDN for Windows headers—to determine the specific data types used within its individual fields.

Answer the questions below

What data type are the PE headers?

STRUCT

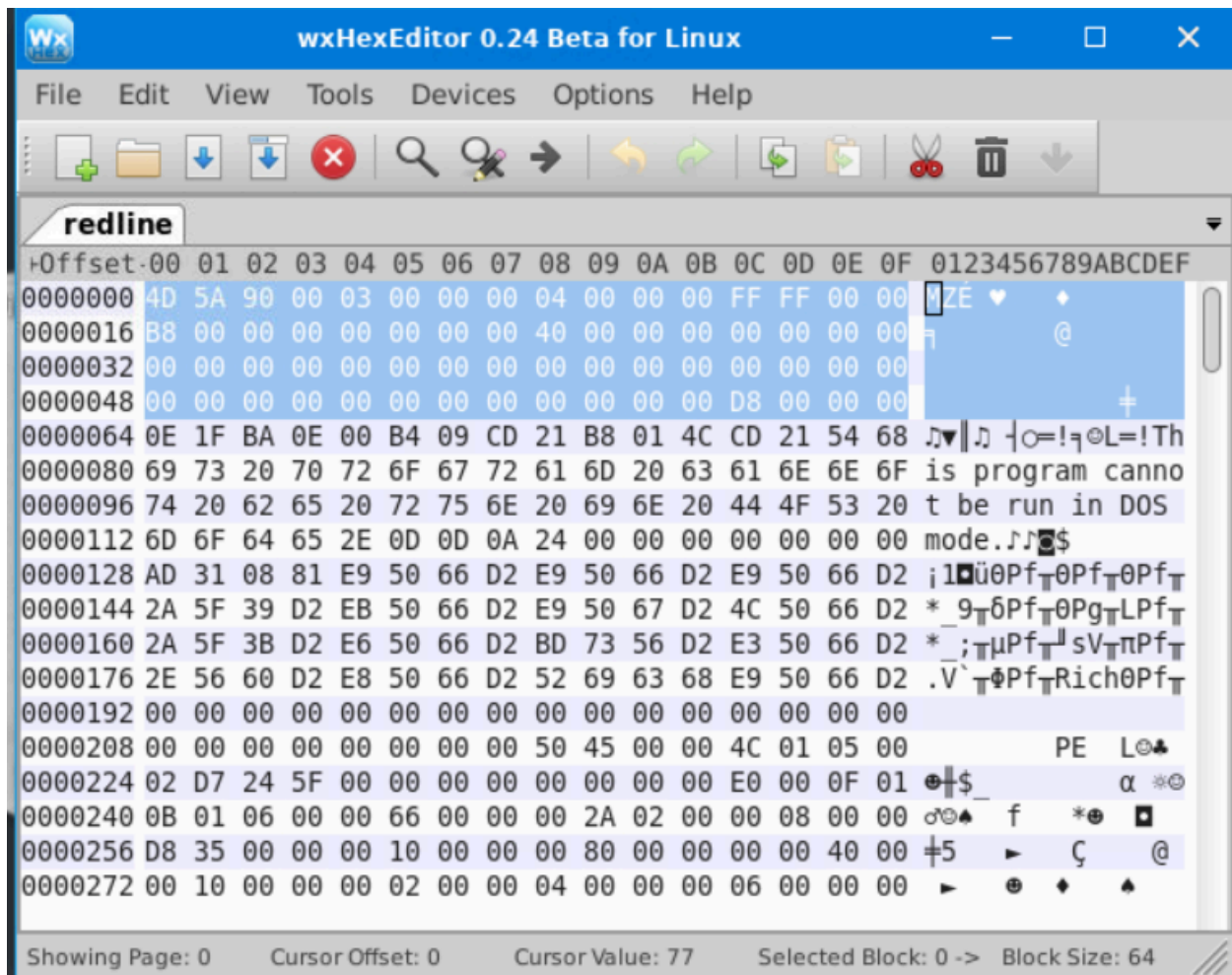
Task 3 - IMAGE_DOS_HEADER and DOS_STUB

pe-tree Desktop/Samples/redline run the PE tree application

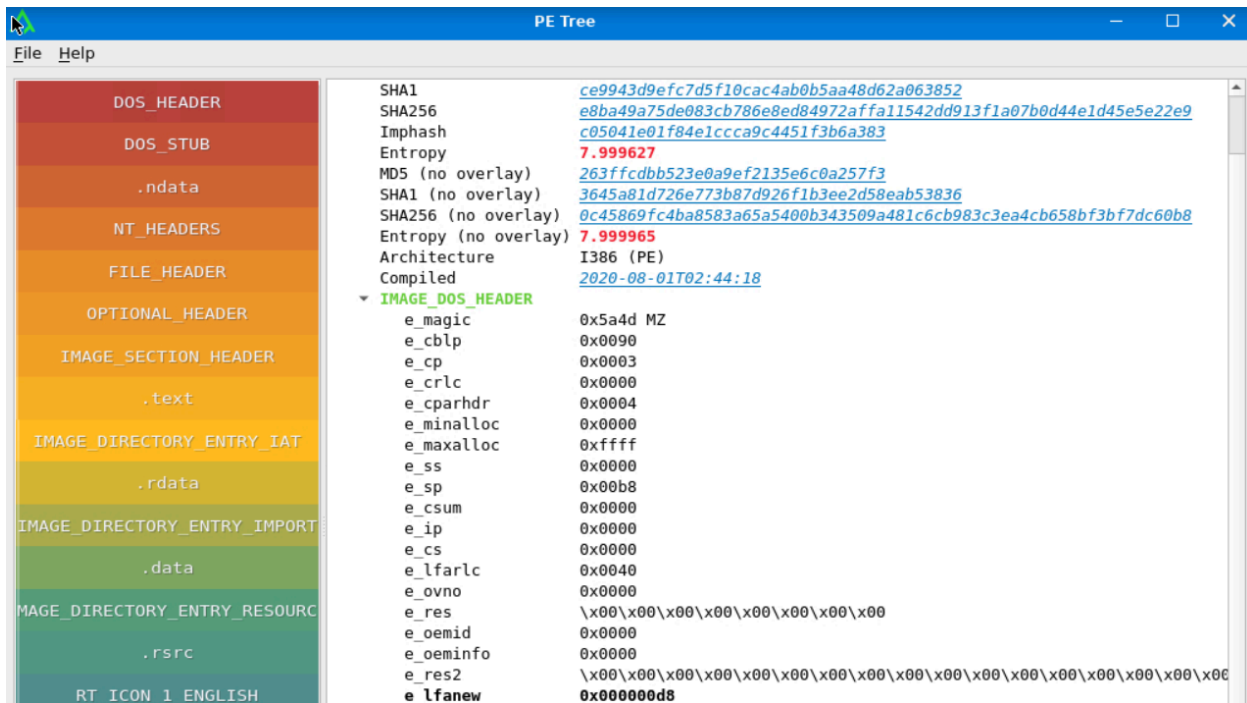


We see the size, hashes, Entropy, architecture, and compiled date of the PE file. This information is not extracted directly from the header; instead, it is calculated or extracted from different parts of the header. The header starts below this information, with the heading `IMAGE_DOS_HEADER`.

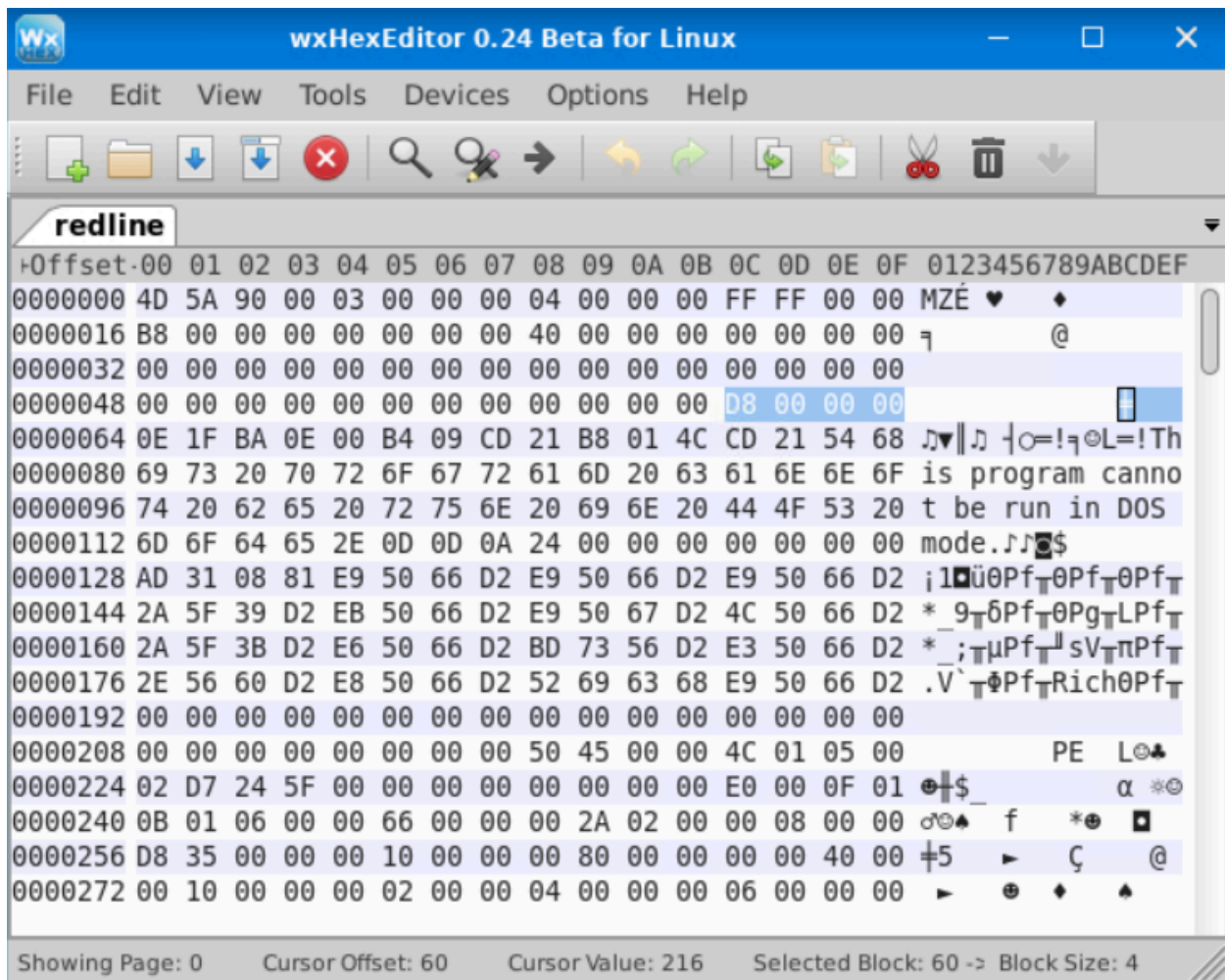
The `IMAGE_DOS_HEADER`: The `IMAGE_DOS_HEADER` consists of the first 64 bytes of the PE file. The below screenshot has the `IMAGE_DOS_HEADER` highlighted in the Hex Editor.



The first two bytes of a Portable Executable file (**4D 5A** , ASCII "**MZ**") serve as a mandatory magic number header—named after architect Mark Zbikowski—that tells the Windows OS the file is an executable format.



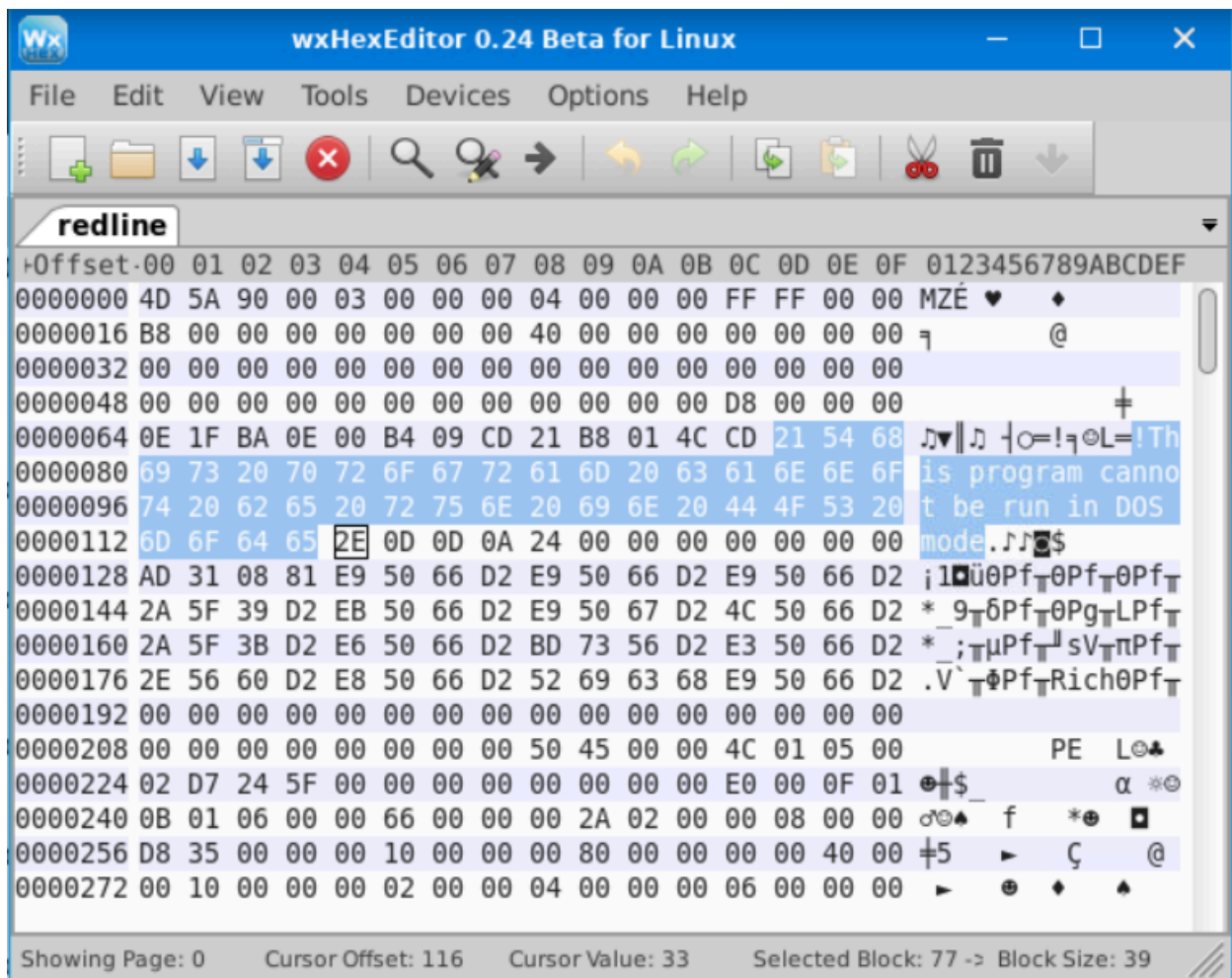
In the `IMAGE_DOS_HEADER`, the signature `e_magic` appears as `0x5a4d` (reversed from raw bytes `4D 5A` due to Intel x86's little-endian architecture), while the final field, `e_lfanew` (value `0x000000d8`), acts as a pointer marking the exact starting file address of the `IMAGE_NT_HEADERS`.



e_ifnew is a 4-byte field located at the very end of the `IMAGE_DOS_HEADER` (at offset `0x3C`) that contains the **file offset (address pointer)** indicating where the `IMAGE_NT_HEADERS` section begins in a Windows Portable Executable (PE) file.

Endianness is the byte-order convention a computer system uses to store multi-byte data in memory—either placing the most significant byte first (Big-Endian) or the least significant byte first (Little-Endian).

The DOS_STUB: The **DOS Stub** is a small, legacy fallback program that displays the message *"!This program cannot be run in DOS mode"* and safely exits if a modern Windows PE executable is run on an incompatible system like MS-DOS. Analysis tools like `pe-tree` can evaluate this specific section's size, hashes, and entropy (measure of data randomness) independently from the rest of the binary.



Answer the questions below

| How many bytes are present in the IMAGE_DOS_HEADER?

64 (The standard IMAGE_DOS_HEADER structure in a Windows PE file is 64 bytes (0x40 bytes) long, spanning from offset 0x00 (e_magic))

▼ zmsuz3pinwl	⚠ 8 warnings
Size	1.2 MB (1247232 bytes)
MD5	1ebbbe268a462d56a389e8e1d06b4945
SHA1	1ecc0b9f380896373e81ed166c34a89bde873b5
SHA256	98c6cf0b129438ec62a628e8431e790b114ba0d82b76e625885ceedef286d6f5
Imphash	829df24b10d39ea3719a931bde7a76d7
Entropy	7.978052
Architecture	I386 (PE)
Compiled	2022-03-09T12:27:49
▼ IMAGE_DOS_HEADER	
e_magic	0x5a4d MZ
e_cblp	0x0090
e_cp	0x0003
e_crlc	0x0000
e_cparhdr	0x0004
e_minalloc	0x0000
e_maxalloc	0xffff
e_ss	0x0000
e_sp	0x00b8
e_csum	0x0000
e_ip	0x0000
e_cs	0x0000
e_lfarlc	0x0040
e_ovno	0x0000
e_res	\x00\x00\x00\x00\x00\x00\x00\x00
e_oemid	0x0000
e_oeminfo	0x0000
e_res2	\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
e_lfanew	0x000000f8

What does MZ stand for?

Mark Zbikowski

In what variable of the IMAGE_DOS_HEADER is the address of IMAGE_NT_HEADERS saved?

e_lfanew

In the attached VM, open the PE file Desktop/Samples/zmsuz3pinwl in pe-tree. What is the address of IMAGE_NT_HEADERS for this PE file?

0x000000f8

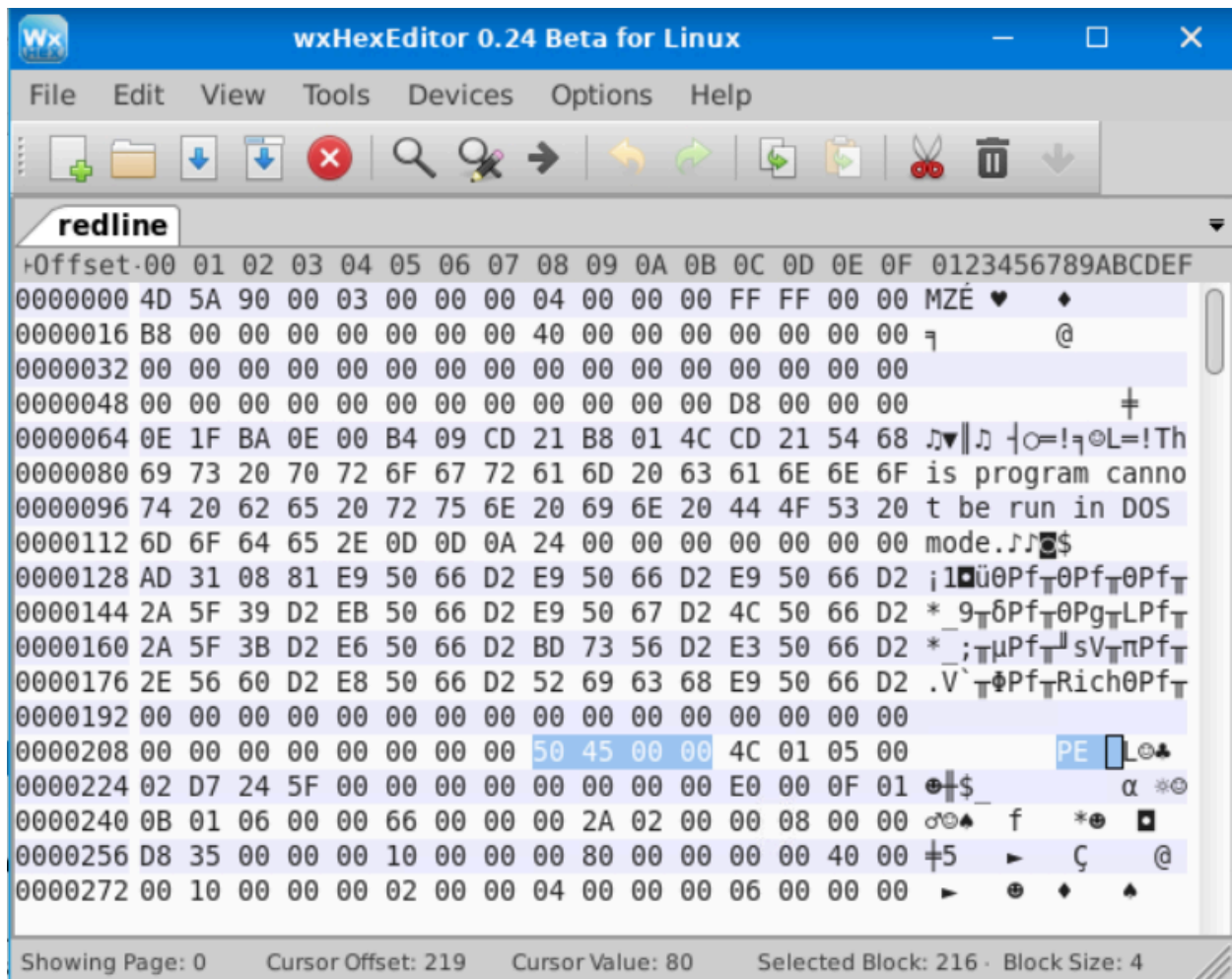
Task 4 - IMAGE_NT_HEADERS

NT_HEADERS: The NT_HEADERS consist of the following:

- Signature
- FILE_HEADER
- OPTIONAL_HEADER

To reach the `IMAGE_NT_HEADERS` starting address specified by `e_lfanew` (e.g., `0x000000D8`), navigate in the Hex Editor using **Ctrl+G (Edit > Go to offset)** with the branch type set to **"From beginning"** and the data type set to **Hex**.

Signature: The `IMAGE_NT_HEADERS` begins with a 4-byte signature (`50 45 00 00` in Hex or **"PE\0\0"** in ASCII) marking its start, and contains two main internal components: the `FILE_HEADER` and the `IMAGE_OPTIONAL_HEADER`.



FILE_HEADER:

▼ redline	
Size	5.7 MB (5985678 bytes)
MD5	ca2dc5a3f94c4f19334cc8b68f256259
SHA1	ce9943d9efc7d5f10cac4ab0b5aa48d62a063852
SHA256	e8ba49a75de083cb786e8ed84972affa11542dd913f1a07b0d44e1d45e5e22
Imphash	c05041e01f84e1ccca9c4451f3b6a383
Entropy	7.999627
MD5 (no overlay)	263ffcd5b523e0a9ef2135e6c0a257f3
SHA1 (no overlay)	3645a81d726e773b87d926f1b3ee2d58eab53836
SHA256 (no overlay)	0c45869fc4ba8583a65a5400b343509a481c6cb983c3ea4cb658bf3bf7dc60
Entropy (no overlay)	7.999965
Architecture	I386 (PE)
Compiled	2020-08-01T02:44:18
▶ IMAGE_DOS_HEADER	
▶ DOS_STUB	
▼ IMAGE_NT_HEADERS	
▼ NT_HEADERS	
Signature	0x00004550 PE
▼ FILE_HEADER	
Machine	0x014c I386
NumberOfSections	0x0005
TimeDateStamp	0x5f24d702 Sat Aug 1 02:44:18 2020 UTC
PointerToSymbolTable	0x00000000
NumberOfSymbols	0x00000000
SizeOfOptionalHeader	0x00e0 224 bytes
Characteristics	0x010f RELOCS_STRIPPED EXECUTABLE_IMAGE LINE_NUMS_STRIPPED
▶ OPTIONAL_HEADER	
▶ IMAGE_SECTION_HEADER	
▶ IMAGE_THUNK_DESCRIPTOR	

As we can see in the above screenshot, the FILE_HEADER has the following fields:

- Machine: Identifies the target architecture (e.g., **i386** for 32-bit Intel).
- NumberOfSections: Specifies the total number of sections in the file (e.g., 5).
- TimeDateStamp: This field contains the time and date of the binary compilation.
- PointerToSymbolTable and NumberOfSymbols: Legacy COFF fields generally unused in standard PE files.
- SizeOfOptionalHeader: Defines the size of the following **IMAGE_OPTIONAL_HEADER** (e.g., 224 bytes).
- Characteristics: Flags key file attributes (e.g., executable, 32-bit compatible, stripped relocation/symbol info).

Answer the questions below

In the attached VM, there is a file Desktop\Samples\zmsuz3pinwl. Open this file in pe-tree. Is this PE file compiled for a 32-bit machine or a 64-bit machine?

32-bit machine (File Header)

NT_HEADERS	
Signature	0x00004550 PE
FILE_HEADER	
Machine	0x014c I386
NumberOfSections	0x0007
TimeDateStamp	0x62289d45 Wed Mar 9 12:27:49 2022 UTC
PointerToSymbolTable	0x00000000
NumberOfSymbols	0x00000000
SizeOfOptionalHeader	0x00e0 224 bytes
Characteristics	0x0103 RELOCS_STRIPPED EXECUTABLE_IMAGE 32BIT_MACHINE

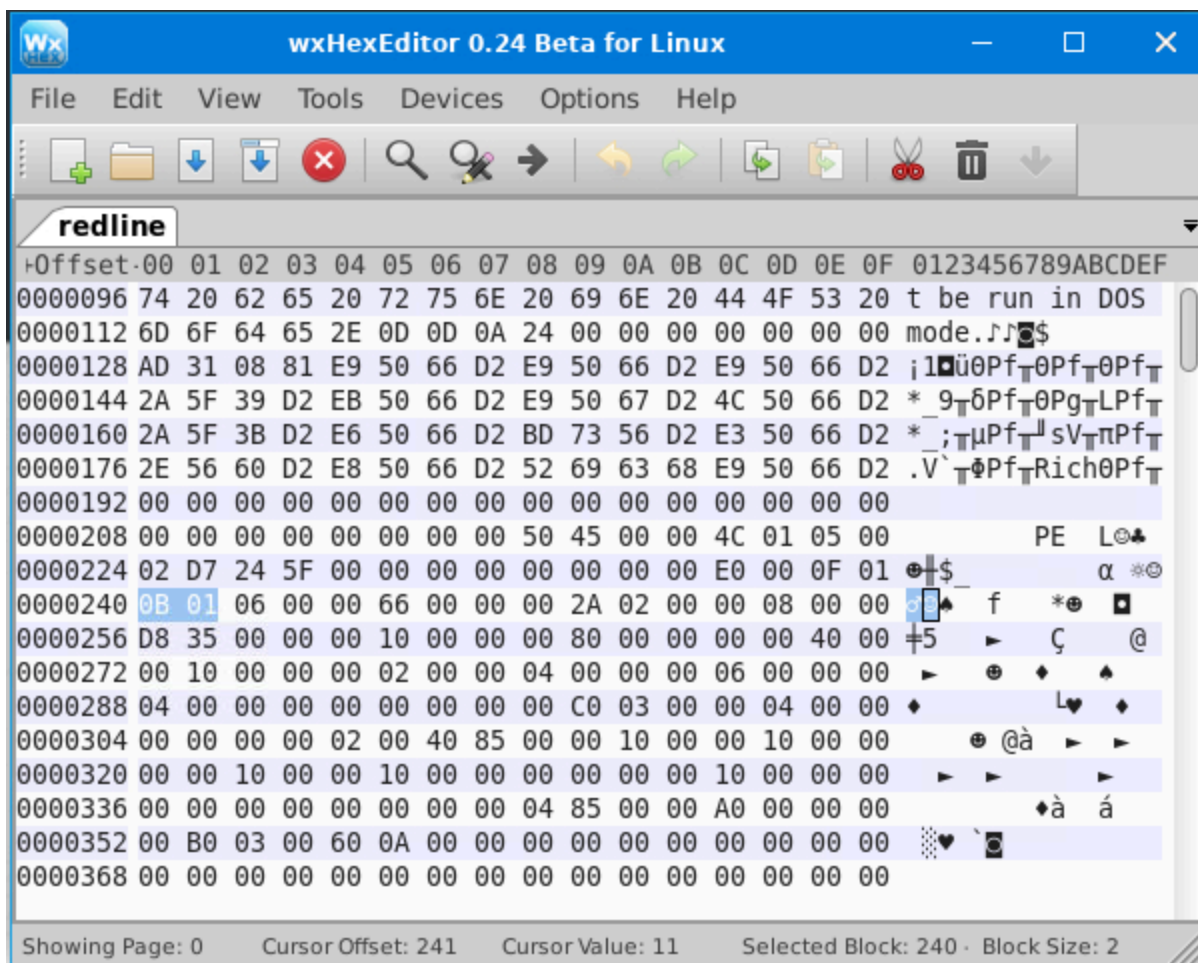
What is the TimeDateStamp of this file?

0x62289d45 Wed Mar 9 12:27:49 2022 UTC

Task 5 - OPTIONAL_HEADER

The OPTIONAL_HEADER is also a part of the NT_HEADERS. It contains some of the most important information present in the PE headers.

In the Hex editor, the OPTIONAL_HEADER starts right after the end of the FILE_HEADER.



Some of the critical fields in the OPTIONAL_HEADER.

- Magic: Identifies binary architecture—0x010B for 32-bit (PE32) and 0x020B for 64-bit (PE32+).
- AddressOfEntryPoint: The RVA (Relative Virtual Address) pointing to the very first instruction executed when the file runs.
- BaseOfCode and BaseOfData: The RVAs marking the start of the code (.text) and data sections in memory.
- ImageBase: The preferred memory address where the OS loads the executable (typically 0x00400000 for .exe files).

- Subsystem: The execution environment required (e.g., `0x0002` for Windows GUI, or CUI/Native).
- DataDirectory: An array of structures pointing to critical tables, such as the Import Address Table (IAT) and Export Address Table (EAT).

Answer the questions below

Which variable from the OPTIONAL_HEADER indicates whether the file is a 32-bit or a 64-bit application?

Magic

What Magic value indicates that the file is a 64-bit application?

0x020B

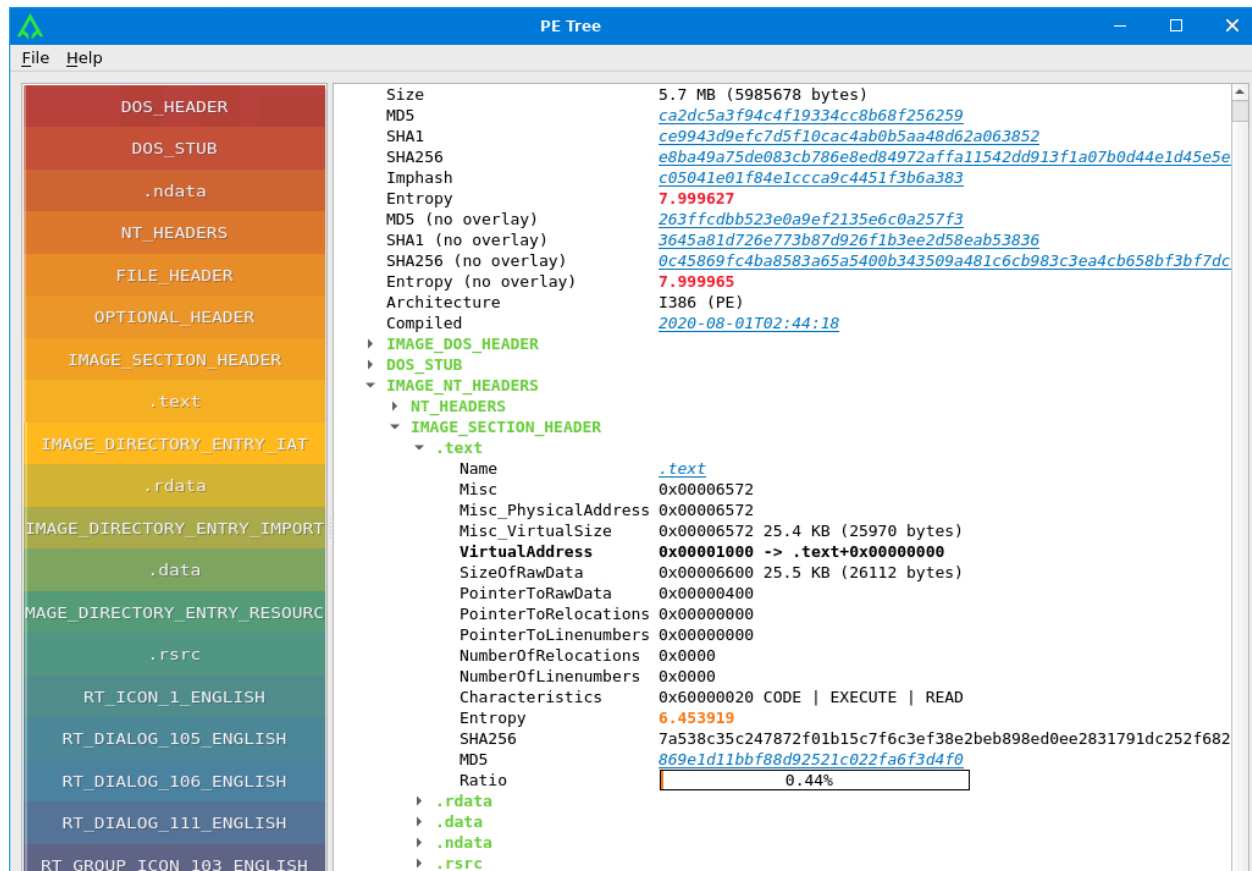
What is the subsystem of the file Desktop\Samples\zmsuz3pinwl?

0x0003 WINDOWS_CUI

OPTIONAL_HEADER	
Magic	0x010b PE
MajorLinkerVersion	0x0e 14.30
MinorLinkerVersion	0x1e
SizeOfCode	0x003f3800 4.0 MB (4143104 bytes)
SizeOfInitializedData	0x00014c00 83.0 KB (84992 bytes)
SizeOfUninitializedData	0x00000000 0 bytes
AddressOfEntryPoint	0x00001000 -> +0x00000000
BaseOfCode	0x00001000
BaseOfData	0x003f5000
ImageBase	0x00400000 -> .rsrc+0x00008000
SectionAlignment	0x00001000
FileAlignment	0x00000200
MajorOperatingSystemVersion	0x0006 Windows Vista/Windows Server 2008
MinorOperatingSystemVersion	0x0000
MajorImageVersion	0x0000
MinorImageVersion	0x0000
MajorSubsystemVersion	0x0006
MinorSubsystemVersion	0x0000
Reserved1	0x00000000
SizeOfImage	0x00469000 4.4 MB (4624384 bytes)
SizeOfHeaders	0x00000400 1.0 KB (1024 bytes)
Checksum	0x0040991b
Subsystem	0x0003 WINDOWS_CUI

Task 6 - IMAGE_SECTION_HEADER

The data that a PE file needs to perform its functions, like code, icons, images, User Interface elements, etc., are stored in different Sections.



- .text: Contains executable code (Read, Execute permissions).
- .data: Contains initialized global and static variables (Read, Write permissions).
- .rdata/.idata: Contains read-only data, including import tables and API references.
- .ndata: The .ndata section contains uninitialized data.
- .reloc: Holds relocation entries used when the binary isn't loaded at its preferred `ImageBase`.
- .rsrc: Stores UI resources like icons, images, strings, and manifest files.

What important information the section headers for each section include:

- VirtualAddress: The RVA (Relative Virtual Address) where the section starts in memory.
- VirtualSize: The size of the section in memory after being loaded and aligned.
- SizeOfRawData: The physical size of the section as it exists on the disk.
- Characteristics: Access flags defining section behavior and permissions (e.g., Read, Write, Execute, Code, Data).

Answer the questions below

| How many sections does the file Desktop\Samples\zmsuz3pinwl have?

7

```

▼ IMAGE_NT_HEADERS
  ▸ NT_HEADERS
  ▼ IMAGE_SECTION_HEADER
    ▸
    ▸
    ▸
    ▸ .rsrc
    ▸
    ▸ .data
    ▸ .adata

```

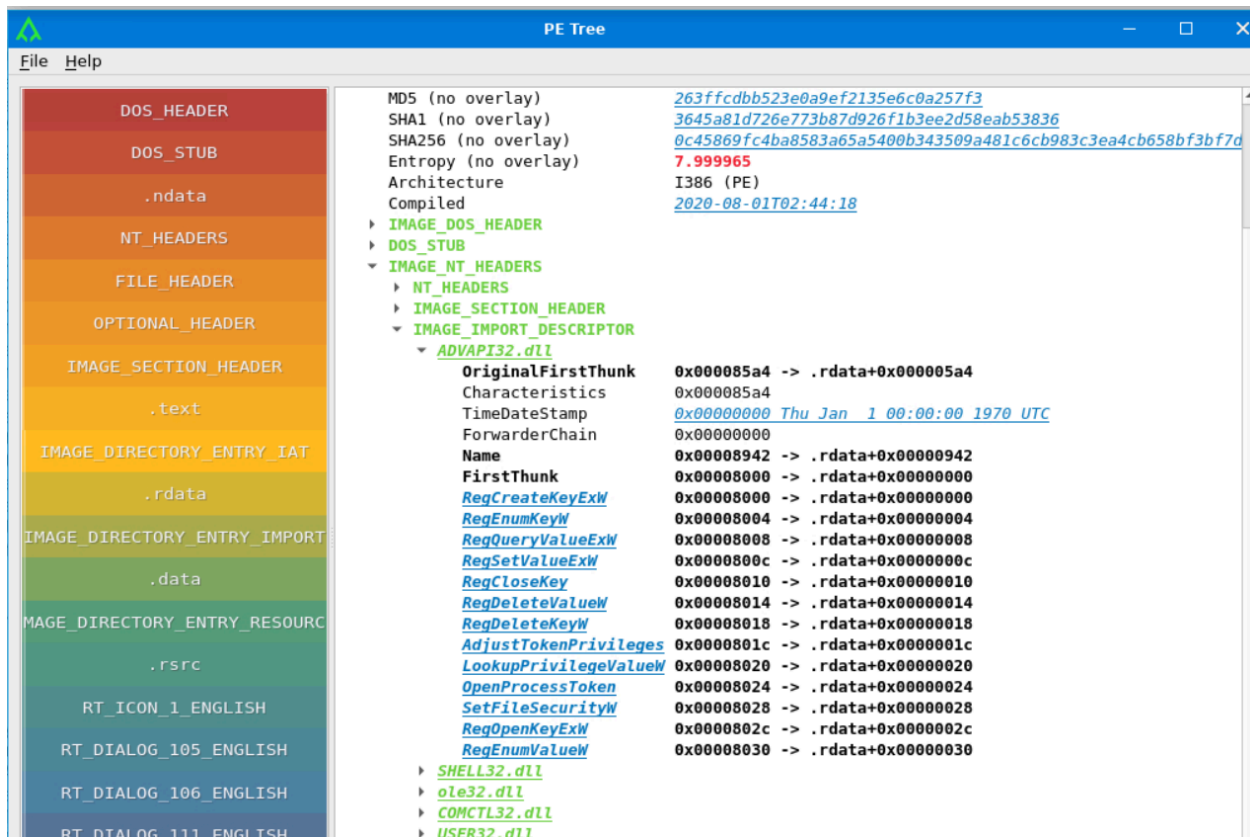
| What are the characteristics of the .rsrc section of the file Desktop\Samples\zmsuz3pinwl

0xe0000040 INITIALIZED_DATA | EXECUTE | READ | WRITE

▼	IMAGE_SECTION_HEADER	
▶		
▶		
▶		
▼	.rsrc	
	Name	.rsrc
	Misc	0x00013000
	Misc_PhysicalAddress	0x00013000
	Misc_VirtualSize	0x00013000 76.0 KB (77824 bytes)
	VirtualAddress	0x003f8000 -> .rsrc+0x00000000
	SizeOfRawData	0x00000400 1.0 KB (1024 bytes)
	PointerToRawData	0x000d4e00
	PointerToRelocations	0x00000000
	PointerToLinenumbers	0x00000000
	NumberOfRelocations	0x0000
	NumberOfLinenumbers	0x0000
	Characteristics	0xe0000040 INITIALIZED_DATA EXECUTE READ WRITE
	Entropy	6.938747
	SHA256	3316da18e71713f02b2fe1e518eed352068a35387e649bac8b411e1
	MD5	08c9e0fc9b79f886b9e3bcb2686d3887
	Ratio	0.00%

Task 7 - IMAGE_IMPORT_DESCRIPTOR

The `IMAGE_IMPORT_DESCRIPTOR` structure details the external Windows APIs imported by a PE file, offering valuable insight into its potential behavior and functionality upon execution.



PE files use dynamic linking to import Windows APIs from DLLs (like `ADVAPI32.dll` or `USER32.dll`), using fields like `OriginalFirstThunk` and `FirstThunk` to build the Import Address Table (IAT), which reveals specific capabilities (e.g., registry modification via `RegCreateKeyExW`).

FirstThunk	0x00008070 -> .rdata+0x00000070
<u>GetExitCodeProcess</u>	0x00008070 -> .rdata+0x00000070
<u>WaitForSingleObject</u>	0x00008074 -> .rdata+0x00000074
<u>GetModuleHandleA</u>	0x00008078 -> .rdata+0x00000078
<u>GetProcAddress</u>	0x0000807c -> .rdata+0x0000007c
<u>GetSystemDirectoryW</u>	0x00008080 -> .rdata+0x00000080
<u>lstrcatW</u>	0x00008084 -> .rdata+0x00000084
<u>Sleep</u>	0x00008088 -> .rdata+0x00000088
<u>lstrcpyA</u>	0x0000808c -> .rdata+0x0000008c
<u>WriteFile</u>	0x00008090 -> .rdata+0x00000090
<u>GetTempFileNameW</u>	0x00008094 -> .rdata+0x00000094
<u>lstrcpmA</u>	0x00008098 -> .rdata+0x00000098
<u>RemoveDirectoryW</u>	0x0000809c -> .rdata+0x0000009c
<u>CreateProcessW</u>	0x000080a0 -> .rdata+0x000000a0
<u>CreateDirectoryW</u>	0x000080a4 -> .rdata+0x000000a4
<u>GetLastError</u>	0x000080a8 -> .rdata+0x000000a8
<u>CreateThread</u>	0x000080ac -> .rdata+0x000000ac
<u>GlobalLock</u>	0x000080b0 -> .rdata+0x000000b0
<u>GlobalUnlock</u>	0x000080b4 -> .rdata+0x000000b4
<u>GetDiskFreeSpaceW</u>	0x000080b8 -> .rdata+0x000000b8
<u>WideCharToMultiByte</u>	0x000080bc -> .rdata+0x000000bc

Answer the questions below

The PE file Desktop\Samples\redline imports the function CreateWindowExW.
From which dll file does it import this function?

User32.dll

USER32.dll	
OriginalFirstThunk	0x00008738 -> .rdata+0x00000738
Characteristics	0x00008738
TimeStamp	0x00000000 Thu Jan 1 00:00:00 1970
ForwarderChain	0x00000000
Name	0x00008e98 -> .rdata+0x00000e98
FirstThunk	0x00008194 -> .rdata+0x00000194
GetClientRect	0x00008194 -> .rdata+0x00000194
EndPaint	0x00008198 -> .rdata+0x00000198
DrawTextW	0x0000819c -> .rdata+0x0000019c
IsWindowEnabled	0x000081a0 -> .rdata+0x000001a0
DispatchMessageW	0x000081a4 -> .rdata+0x000001a4
wsprintfA	0x000081a8 -> .rdata+0x000001a8
CharNextA	0x000081ac -> .rdata+0x000001ac
CharPrevW	0x000081b0 -> .rdata+0x000001b0
MessageBoxIndirectW	0x000081b4 -> .rdata+0x000001b4
GetDlgItemTextW	0x000081b8 -> .rdata+0x000001b8
SetDlgItemTextW	0x000081bc -> .rdata+0x000001bc
GetSystemMetrics	0x000081c0 -> .rdata+0x000001c0
FillRect	0x000081c4 -> .rdata+0x000001c4
AppendMenuW	0x000081c8 -> .rdata+0x000001c8
TrackPopupMenu	0x000081cc -> .rdata+0x000001cc
OpenClipboard	0x000081d0 -> .rdata+0x000001d0
SetClipboardData	0x000081d4 -> .rdata+0x000001d4
CloseClipboard	0x000081d8 -> .rdata+0x000001d8
IsWindowVisible	0x000081dc -> .rdata+0x000001dc
CallWindowProcW	0x000081e0 -> .rdata+0x000001e0
GetMessagePos	0x000081e4 -> .rdata+0x000001e4
CheckDlgButton	0x000081e8 -> .rdata+0x000001e8

Task 8 - Packing and Identifying packed executables

Packers obfuscate a PE file's data and structure to prevent static analysis and reverse engineering, using an unpacking routine at runtime to restore and execute the original code—a technique used both by legitimate developers to prevent piracy and by malware authors to evade detection.

From Section Headers: Packed PE files can be identified through section header indicators such as missing or unusual section names, high entropy levels near 8, combined Read-Write-Execute (RWX) permissions, and a `Misc_VirtualSize` significantly larger than its `SizeOfRawData`.

`pecheck zmsuz3pinwl` It is a command-line instruction to run the `pecheck` static analysis tool against the target binary `zmsuz3pinwl` to extract its PE header structure, sections, and metadata.

From Import functions: Packed executables often feature a minimal import table—frequently limited to functions like `GetProcAddress`, `GetModuleHandleA`, and `LoadLibraryA`—which provide the essential runtime capabilities needed to dynamically load libraries and unpack the payload.

Answer the questions below

Which of the files in the attached VM in the directory Desktop\Samples seems to be a packed executable?

zmsuz3pinwl

Task 9 - Conclusion

- What is a PE header
- What are the different parts of the PE header
- How to read the information from the PE header
- Identify packed executables using the PE header